Coral Reef Health Assessment: Steps 1-9 Review Guide

English version | Concise notes for presentation review

Purpose: concise review notes for presentation preparation. Use this document to understand the full pipeline from dataset collection to ReefGuide deployment.

Pipeline Overview

Step	Process	Main Purpose	Why It Matters
Step 1	Data Acquisition	Collect coral image file paths and assign numeric labels.	Creates a clean input list before splitting and training.
Step 2	Data Splitting	Split the dataset into train, validation, and test sets.	Prevents data leakage and gives a fair final test.
Step 3	Preprocessing and Augmentation	Standardize images and create training variation.	Improves model robustness against camera angle, distance, lighting, and water color changes.
Step 4	Oversampling and Class Weighting	Handle imbalanced classes and difficult examples.	Stops the model from favoring majority classes and improves attention to rare coral conditions.
Step 5	Model Development and Training	Train an EfficientNet-B0 classifier for three coral health classes.	Combines transfer learning efficiency with stable ensemble predictions.
Step 6	SWA Optimization	Improve model generalization using Stochastic Weight Averaging.	Moves the model toward flatter minima, which usually performs more consistently on unseen data.
Step 7	Final Evaluation and TTA	Evaluate final performance on unseen test images.	Produces more stable predictions and better-calibrated confidence values.
Step 8	Grad-CAM Explainability	Show which image regions influenced the model decision.	Makes the prediction easier to validate and explain scientifically.
Step 9	ReefGuide Deployment	Deploy the trained model into an interactive Flask and React web application.	Turns the trained model into a usable tool for researchers, divers, and public users.

Step-by-Step Review Notes

Step 1: Data Acquisition

Purpose	Collect coral image file paths and assign numeric labels.
Key Method	- Read Healthy, Bleached, and Dead folders. - Accept only .png, .jpg, and .jpeg files. - Store file paths, labels, and relative file names.
Reason	Creates a clean input list before splitting and training.
Presentation Point	The model cannot learn from folder names directly, so each folder is mapped to a class number.

Code reference: train_v4_robust.py - collect_file_paths

Step 2: Data Splitting

Purpose	Split the dataset into train, validation, and test sets.
Key Method	- Use stratified train_test_split. - Keep class ratios balanced across splits. - Save or reload split_info_v3.json for reproducibility.
Reason	Prevents data leakage and gives a fair final test.
Presentation Point	The test set stays unseen until final evaluation, so reported accuracy is more trustworthy.

Code reference: train_v4_robust.py - split_dataset

Step 3: Preprocessing and Augmentation

Purpose	Standardize images and create training variation.
Key Method	- Resize images to 224 x 224. - Convert OpenCV BGR images to RGB. - Apply rotation, shift, zoom, brightness, hue, saturation, and mixup augmentation.
Reason	Improves model robustness against camera angle, distance, lighting, and water color changes.
Presentation Point	Preprocessing standardizes input; augmentation teaches the model not to memorize exact images.

Code reference: train_v4_robust.py - load_images, prepare_set, get_augmenter, AugmentedMixupGenerator

Step 4: Oversampling and Class Weighting

Purpose	Handle imbalanced classes and difficult examples.
Key Method	- Duplicate known hard examples during training. - Oversample Dead examples more strongly. - Use balanced class weights and increase Dead-class penalty by 1.3x.
Reason	Stops the model from favoring majority classes and improves attention to rare coral conditions.
Presentation Point	This is important because Dead coral has fewer examples, so mistakes on that class must carry more weight.

Code reference: train_v4_robust.py - prepare_training_data, class_weight

Step 5: Model Development and Training

Purpose	Train an EfficientNet-B0 classifier for three coral health classes.

Key Method	- Use EfficientNet-B0 pretrained on ImageNet. - Freeze early layers and fine-tune the top 100 layers. - Add pooling, dropout 0.4, and a 3-class softmax output. - Train a 5-seed ensemble using cosine learning-rate decay.
Reason	Combines transfer learning efficiency with stable ensemble predictions.
Presentation Point	EfficientNet-B0 is lightweight enough for deployment but still accurate for image classification.

Code reference: train_v4_robust.py - build_model, cosine_decay

Step 6: SWA Optimization

Purpose	Improve model generalization using Stochastic Weight Averaging.
Key Method	- Collect weights from the last 5 epochs. - Average the weights mathematically. - Save final SWA weights after training.
Reason	Moves the model toward flatter minima, which usually performs more consistently on unseen data.
Presentation Point	Instead of trusting one final epoch, SWA averages several late-stage model states.

Code reference: train_v4_robust.py - SWACallback

Step 7: Final Evaluation and TTA

Purpose	Evaluate final performance on unseen test images.
Key Method	- Use held-out test data. - Run Test-Time Augmentation with original and flipped images at 224 and 256 center-crop scale. - Average predictions across 5 ensemble models and 4 TTA views. - Apply temperature scaling with T = 0.441.
Reason	Produces more stable predictions and better-calibrated confidence values.
Presentation Point	Each image receives multiple predictions, then the system averages them for a stronger final decision.

Code reference: train_v4_robust.py - predict_with_tta; app.py - temperature_scale_from_probs

Step 8: Grad-CAM Explainability

Purpose	Show which image regions influenced the model decision.
Key Method	- Use Grad-CAM on EfficientNet top_conv. - Average heatmaps across the ensemble. - Use eigen-smooth denoising for cleaner visual output. - Overlay the heatmap on the original coral image.
Reason	Makes the prediction easier to validate and explain scientifically.
Presentation Point	Grad-CAM helps confirm whether the model focuses on coral tissue rather than irrelevant background.

Code reference: train_v4_robust.py - make_gradcam_heatmap

Step 9: ReefGuide Deployment

Purpose	Deploy the trained model into an interactive Flask and React web application.
Key Method	- Expose /api/predict for image classification. - Return prediction, confidence, probabilities, Grad-CAM image, original image, and uncertainty flag. - Integrate ReefGuide chatbot using Gemini 2.5 Flash through google-genai. - Support comparison between ensemble and baseline model modes.
Reason	Turns the trained model into a usable tool for researchers, divers, and public users.
Presentation Point	Deployment makes the project practical: users can upload an image, see the prediction, view Grad-CAM, and ask ReefGuide for explanation.

Code reference: 04_Web_Application/app.py - predict, generate_gemini_reply

Presentation Review Checklist

Topic	What to Say	Keep It Concise
Problem	Classify coral reef health into Healthy, Bleached, and Dead.	Explain the real-world conservation value.
Data	Images are collected, filtered, labelled, and split safely.	Mention no data leakage.
Training	EfficientNet-B0 is fine-tuned with augmentation, class balancing, ensemble seeds, and SWA.	Focus on reliability, not every code detail.
Evaluation	TTA and temperature scaling make predictions more stable and confidence more meaningful.	Say each image is tested in multiple views.
Explainability	Grad-CAM shows where the model focuses.	Use this to build trust.
Deployment	Flask and React expose prediction, Grad-CAM, and ReefGuide chatbot support.	End with practical usability.